Machine learning message-passing for the scalable decoding of QLDPC codes

Williamdepig

Note by Williamdepig

November 28, 2025

Contents

1. Synopsis

1.1. Problem

BP does not have a threshold for the surface code and fails to decode most of the time, leading to the necesity of a second stage decoder(OSD), which increases the decoding runtime. Need to find a (machine learning) method that can successfully decode QLPDC codes in a single stage.

1.2. Solution

Astra: The decoder is based on graph neural networks (GNN) which learn a message-passing algorithm(BP) for decoding. 3

- Traditional neural network decoder: *the entire syndrome → the entire error mode → logical error happened*
- Astra: *local function → message passing algorithm → solving sudoku problem and predict node state → logical error happened*

1.3. Achievement

1. works on any family of codes that can be represented by Tanner graphs;
2. is capable of transfer learning(scalable);
3. has faster runtimes compared to BP + OSD while improving on accuracy.
4. higher PER threshold, lower LER

2. Principle

Core logic: The degree of the nodes does not change with increasing QECC distance. Once learnt, the functions can be reused for interpolated and extrapolated decoding, as well as extrapolated training.

2.1. Local functions

1. f: how to send messages between nodes. multi-layer perceptron(**MLP**).MLP_1 for sending message.
2. g: how to combine the messages and update the node status. Another NN, a gated-recurrent unit (**GRU**) to learn the g function.
3. r: how to compute the final outputs from the node states.MLP_2 for output.

2.2. Tanner graph

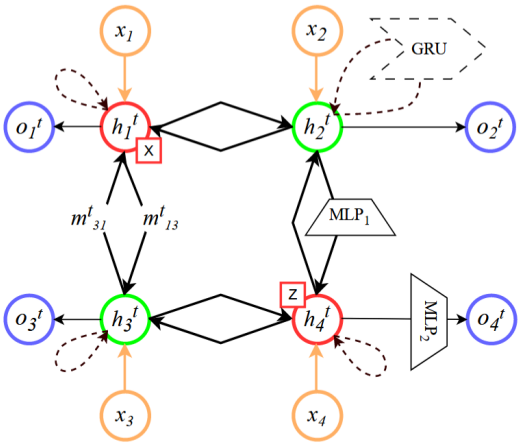


FIG. 2. The network architecture of Astra is based on the Tanner graph of the code. The green circles are data nodes and red are check nodes (one X check, and one Z check). We are only showing 2 edges per node for simplicity. The node's hidden state is h_i^t, the inputs x_i are orange, and the outputs o_i^t are purple. Messages, e.g. m_{13}^t are exchanged between the nodes based on the Tanner graph connectivity shown as bold edges and calculated using MLP_1 **f** i.e. one MLP for all the messages. The hidden node states, h_i^t, are updated after each iteration indicated by a dashed line using one Gated Recurrent Unit (GRU) for all hidden states. Final outputs o_i^t are calculated using another MLP i.e. one MLP_2 to calculate all outputs from hidden node states. This is illustrated formally in the methods section of main text.

2.3. Training

2.3.1. Loss function

To overcome the challenges of training in the presence of degeneracy, it uses a loss function which consists of two parts:

1. Cross entropy loss
2. NBP Loss:

$$\begin{aligned} H^T e_{\text{total}} &= 0 \bmod 2 \\ e_{\text{total}} &= \left(e_{\text{actual}} + e_{\text{predicted}}\right)\%2 \end{aligned} \tag{1}$$

2.3.2. Extrapolated Training

We tested extrapolation training for d_9, in multiple trials with exactly the same hyperparameters training d_9 from scratch took > 90 epochs whereas training d_9 from d_7 took on average 30 epochs to achieve the best LER.

2.4. Defects

1. How to implement multi-rounds continuous decoding for circuit-level error model?
2. Sub-optimality of Extrapolation: $d \rightarrow$ at most $\frac{d-1}{2}$ errors
3. Single scheduling strategy: flooding.
   - serial scheduling better?

3. Relay-BP

3.1. Problem

qLDPC decoder standard:

1. *Flexible*: Decodes a wide range of qLDPC circuits.
2. *Accurate*: Has the ability to achieve the low logical error rates required by logical computations.
3. *Compact*: Has a small footprint on an FPGA.
4. *Fast*: Avoids the backlog problem by processing syndromes at their production rate.

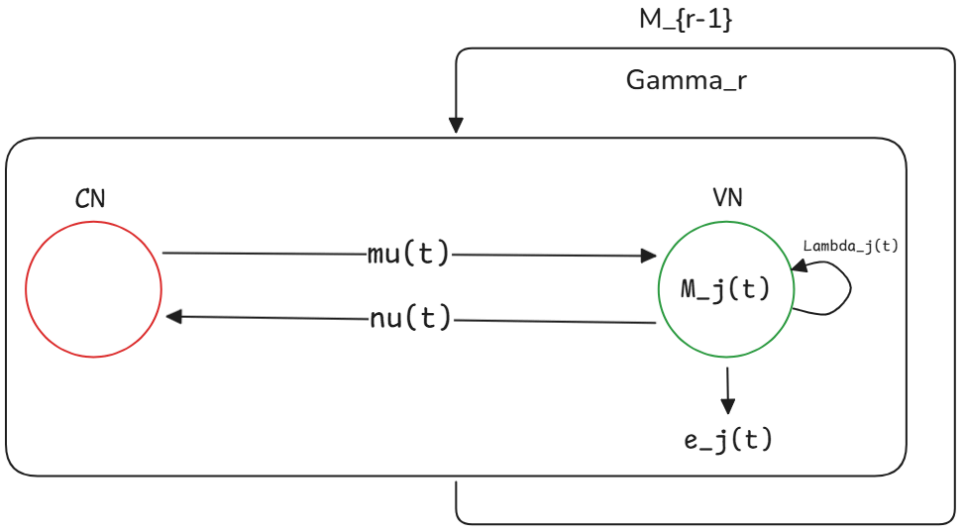
Decoder type	Flexible	Accurate	Compact	Fast
Matching	✗	✓	?	✓
Union-Find	✗	✓	✓	✓
BP/Mem-BP	✓	✗	✓	✓
BP–OSD	✓	○	✗	✗
Clustering-based	✓	○	?	○
Search-based	✓	✓	?	○
Relay–BP	✓	✓	✓	✓

TABLE I. Selected decoder types given our requirements: flexible (applicable to any qLDPC code), accurate, compact (FPGA implementation), and fast. A circle is between a cross and a tick, and a question mark requires further research.

3.2. Solution

- **Mem-BP**: dampen oscillations in BP
- **DMem-BP**(disordered memory strength distributions): improve convergence
- **Relay ensembling technique**: successive DMem-BP runs are initialized with the previous run's final marginals
- **Negative memory strengths**: escape from trapping sets

3.3. Technique details



3.3.1. DMem-BP

In tanner graph, denote messages between check node i and error node j as $\mu_{i\to j}$ and $\nu_{j\to i}$, each passing a local log-likelihood belief of whether an error j occurred (negative) or not (positive).:

$$\mu_{i\to j}(t) = \kappa_{i,j}(t)(-1)^{\sigma_i} \min_{j'\in\mathcal{N}(i)\setminus\{j\}} \left|\nu_{j'\to i}(t-1)\right|, \tag{2}$$

where $k_{i,j}(t) = \operatorname{sgn}\left\{\prod_{j'\in\mathcal{N}(i)\setminus\{j\}} \nu_{j'\to i}(t-1)\right\}$ and $\mathcal{N}(j)$ means the neighbors of node j.

$$\nu_{j\to i}(t) = \Lambda_j(t) + \sum_{i'\in\mathcal{N}(j)\setminus\{i\}} \mu_{i'\to j}(t) \tag{3}$$

For error nodes, we have another value named marginal,defined as:

$$M_j(t) = \Lambda_j(t) + \sum_{i'\in\mathcal{N}(j)} \mu_{i'\to j}(t) \tag{4}$$

and the bias $\Lambda_j(t)$ can be updated via the equation:

$$\Lambda_j(t) = (1-\gamma_j)\Lambda_j(0) + \gamma_j M_j(t-1). \tag{5}$$

Parameter initialization and stopping:

- $\Lambda_j(0) = \nu_{j\to i}(0) = \log\left(\frac{1-p_j}{p_j}\right)$.
- $M_j(0) = M_j$ is set by user input.
- $\Gamma = \{\gamma_j\}_{j\in[N]}$ is memory strength parameter set. The special cases of all γ_j values either zero or a constant between zero and one recover the standard BP and Mem-BP algorithms.
- $\hat{e}_j(t) = \mathrm{HD}\big(M_j(t)\big)$, for $\mathrm{HD}(x) = \frac{1}{2}(1-\operatorname{sgn}(x))$, denotes the hard decision of error node. If $\boldsymbol{H}\hat{\boldsymbol{e}}(t) = \boldsymbol{\sigma}$, the algorithm is considered to have converged. Otherwise we move on until the max iteralions $t = T$.
- $w(\hat{\boldsymbol{e}}) = \sum_j \hat{e}_j \log\left(\frac{1-p_j}{p_j}\right)$, solution weight.

3.3.2. Relay-BP-S

- number of solution sought S,
- maximum number of relay legs R,
- maximum number of iterations for each leg T_r,
- memory strengths for each leg $\mathbf{\Gamma_r} = \{\gamma_j(r)\}_{j\in[N]}$.

Core:

- The r^{th} leg's marginals are initialized with the $(r-1)^{\text{th}}$ legs final marginals.
- Each leg stops upon finding a solution or reaching the iteration limit T_r.
- The algorithm stops when R legs have executed or S solutions have been found.
- Return the lowest-weight solution: $w(\hat{\boldsymbol{e}}) = \sum_j \hat{e}_j \log\!\left(\frac{1-p_j}{p_j}\right)$.

Algorithm 1: Relay-BP-S decoder for quantum LDPC codes

procedure RELAY-BP-S($\boldsymbol{H}$,$\boldsymbol{\sigma}$,$\boldsymbol{p}$,S,R,T_r,$\mathbf{\Gamma_r}$)
  $\lambda_j, M_j(0) \leftarrow \log\frac{1-p_j}{p_j}, r \leftarrow 0, s \leftarrow 0, \hat{\boldsymbol{e}} \leftarrow \emptyset, \omega_{\hat{\boldsymbol{e}}} \leftarrow \infty$
  for $r \le R$ **do**
    // Run DMem-BP
    $\Lambda_j(0) \leftarrow \nu_{j\to i}(0) \leftarrow \lambda_j$
    for $t \le T_r$ **do**
      $\Lambda_j(t) \leftarrow \big(1-\gamma_j(r)\big)\Lambda_j(0) + \gamma_j(r) M_j(t-1)$
      Compute $\mu_{i\to j}(t)$ // via Eq. (1)
      Compute $\nu_{j\to i}(t)$ // via Eq. (2)
      Compute $M_j(t)$ // via Eq. (3)
      $\hat{e}_j(t) \leftarrow \text{HD}(M_j(t))$
      if $\boldsymbol{H}\hat{\boldsymbol{e}}(t) = \boldsymbol{\sigma}$ **then**
        // BP converged
        $\omega_r \leftarrow w(\hat{\boldsymbol{e}}) = \sum_j \hat{e}_j \lambda_j$
        $s \leftarrow s+1$
        if $\omega_r < \omega_{\hat{\boldsymbol{e}}}$ **then**
          $\hat{\boldsymbol{e}} \leftarrow \hat{\boldsymbol{e}}(t)$
          $\omega_{\hat{\boldsymbol{e}}} \leftarrow \omega_r$
        end
        break; // Continue to next leg
      end
      $t \leftarrow t+1$
    end
    if $s = S$ **then**
      break; // Found enough solutions
    end
    // Reuse final marginals for the next leg
    $M_j(0) \leftarrow M_j(t)$
    $r \leftarrow r+1$
  end
  return $(s>0), \hat{\boldsymbol{e}}$;
end

4. Improved GNN?

4.1. 以 **Relay-BP** 为主体

- 考虑改善 Memory selection 机制，用 NN 代替随机选择
- 比如:
  1. Input: $\boldsymbol{\sigma} + M_j(t)$
  2. Structure: several layers of GCN
  3. Output: $\boldsymbol{\Gamma}_r = \{\gamma_j\}$
  4. loss
- 预期: 虽然增加了用 NN 选择 memory strength 的过程，但是预测能大幅减少 Relay-BP 找到正确解所需的 Relay 次数

4.2. 以 **Astra** 为主体

- 消息传递过程，CN 固定使用 Min-Sum 规则(XOR)，而 VN 则保留使用 MLP 来学习
- 保留使用 GRU 来更新节点本轮状态 h_t，而非引入 Memory 机制
- 考虑混合架构，并引入 Relay 机制？如果没有 Memory，引入 relay 没有意义，相当于增加 BP 的 iteration 次数